```
import { useState, useCallback } from "react";

const LEAGUE_XGF_AVG  = 2.55;
const HOME_XG_BOOST   = 0.10;
const LEAGUE_HD_RATIO = 0.35;
const LEAGUE_XSV_PCT  = 0.910;
const MIN_GAMES       = 5;
const ROLLING_W       = 10;
const GOALIE_WINDOW   = 5;
const STAKE           = 10;
const JUICE           = 110;

// ── CSV parser ──────────────────────────────────────────────────
function parseCSV(text) {
  const lines = text.split(/\r?\n/);
  if (!lines.length) return [];
  const headers = lines[0].split(",").map(h => h.trim().replace(/^"|"$/g, ""));
  const rows = [];
  for (let i = 1; i < lines.length; i++) {
    const line = lines[i].trim();
    if (!line) continue;
    const vals = []; let cur = "", inQ = false;
    for (const ch of line) {
      if (ch === '"') { inQ = !inQ; continue; }
      if (ch === "," && !inQ) { vals.push(cur.trim()); cur = ""; continue; }
      cur += ch;
    }
    vals.push(cur.trim());
    if (vals.length < headers.length / 2) continue;
    const row = {};
    headers.forEach((h, j) => { row[h] = vals[j] ?? ""; });
    rows.push(row);
  }
  return rows;
}

const sf = (v, d = 0) => { const n = parseFloat(v); return isNaN(n) ? d : n; };

// ── Goalie xSV% index ──────────────────────────────────────────
// Loads MoneyPuck goalie game-by-game CSV (position=G, situation=all)
// Columns used: playerTeam, gameId, gameDate, xGoals (= xGA faced),
//               goals (= GA), highDangerxGoals, ongoal (shots faced)
// Derives per-start: xSvPct, xSvDelta (GSAE), lambdaMod
```

```
function buildGoalieIndex(rows) {
  const byTeam = {};
  for (const r of rows) {
    if (r.situation !== "all") continue;
    if ((r.position || "").toUpperCase() !== "G") continue;
    const team = r.playerTeam || r.team;
    const gid  = r.gameId || r.game_id;
    if (!team || !gid) continue;

    const xGA   = sf(r.xGoals);
    const ga    = sf(r.goals);
    const shots = sf(r.ongoal);
    const hdXGA = sf(r.highDangerxGoals);

    const xSvPct     = shots > 0 ? 1 - (xGA / shots) : LEAGUE_XSV_PCT;
    const actualSvPct = shots > 0 ? 1 - (ga / shots)  : LEAGUE_XSV_PCT;
    // GSAE: goals saved above expected (positive = goalie outperformed)
    const xSvDelta   = xGA > 0 ? xGA - ga : 0;

    if (!byTeam[team]) byTeam[team] = [];
    byTeam[team].push({
      gameId: gid, date: sf(r.gameDate || r.date),
      xGA, ga, shots, hdXGA, xSvPct, actualSvPct, xSvDelta,
    });
  }
  for (const t of Object.keys(byTeam)) {
    byTeam[t].sort((a, b) => a.date - b.date);
  }
  return byTeam;
}

function goalieForm(goalieIndex, team, beforeGameId, window = GOALIE_WINDOW) {
  const starts = goalieIndex?.[team];
  if (!starts?.length) return null;
  const idx = starts.findIndex(s => s.gameId === beforeGameId);
  if (idx < 2) return null;
  const prior = starts.slice(Math.max(0, idx - window), idx);
  if (!prior.length) return null;

  const avg = key => prior.reduce((s, g) => s + (g[key] || 0), 0) / prior.length;
  const xSvPct   = avg("xSvPct");
  const xSvDelta = avg("xSvDelta");

  const recent = prior.slice(-3);
  const early  = prior.slice(0, -3);
  const recentGSAE = recent.reduce((s, g) => s + g.xSvDelta, 0) / recent.length;
  const earlyGSAE  = early.length ? early.reduce((s, g) => s + g.xSvDelta, 0) / e
```

```
  const formDir    = recentGSAE > earlyGSAE + 0.05 ? 1 : recentGSAE < earlyGSAE -

  // lambdaMod: positive GSAE → opposing team's lambda decreases
  // Calibration: 0.1 GSAE ≈ 0.025 lambda reduction
  const lambdaMod = Math.max(-0.5, Math.min(0.5, xSvDelta * 0.25));

  return { n: prior.length, xSvPct, xSvDelta, formDir, lambdaMod,
           recentGSAE: Math.round(recentGSAE * 100) / 100 };
}

// ── Poisson ──────────────────────────────────────────────────────
const Poisson = {
  pmf(k, lam) {
    if (lam <= 0) return k === 0 ? 1 : 0;
    let lp = -lam + k * Math.log(lam);
    for (let i = 2; i <= k; i++) lp -= Math.log(i);
    return Math.exp(lp);
  },
  winProbs(lH, lA, maxG = 12) {
    let pH = 0, pA = 0, pT = 0;
    for (let h = 0; h <= maxG; h++) for (let a = 0; a <= maxG; a++) {
      const p = this.pmf(h, lH) * this.pmf(a, lA);
      if (h > a) pH += p; else if (a > h) pA += p; else pT += p;
    }
    return { pHomeWin: pH + pT * 0.52, pAwayWin: pA + pT * 0.48 };
  },
  totalProbs(lH, lA, line) {
    const lT = lH + lA;
    let pU = 0;
    for (let k = 0; k <= Math.floor(line - 0.01); k++) pU += this.pmf(k, lT);
    return { pOver: Math.max(0, 1 - pU), pUnder: Math.min(1, pU), expectedTotal:
  },
};

// ── Team profiles ────────────────────────────────────────────────
function buildTeamProfiles(rows) {
  const byTeam = {};
  for (const r of rows) {
    const t = r.team || r.playerTeam; if (!t) continue;
    if (!byTeam[t]) byTeam[t] = [];
    byTeam[t].push(r);
  }
  for (const t of Object.keys(byTeam)) {
    byTeam[t].sort((a, b) => sf(a.gameDate || a.date) - sf(b.gameDate || b.date))
  }
  return byTeam;
}
```

```
function rollingTeamStats(games, upToIdx, window = ROLLING_W) {
  const start = Math.max(0, upToIdx - window);
  const slice = games.slice(start, upToIdx);
  if (!slice.length) return null;
  const n = slice.length;
  const avg = key => { const v = slice.map(g => sf(g[key])); return v.reduce((a,

  const xGF   = avg("xGF_5v5") || avg("xGoals") || LEAGUE_XGF_AVG;
  const xGA   = avg("xGA_5v5") || 0;
  const hdXGF = avg("hdXGF_5v5") || avg("highDangerxGoals") || 0;
  const hdXGA = avg("hdXGA_5v5") || 0;
  const xGF_pp = avg("xGF_pp") || 0;

  const xGFkey = g => sf(g.xGF_5v5) || sf(g.xGoals) || LEAGUE_XGF_AVG;
  const xGAkey = g => sf(g.xGA_5v5) || 0;

  const half = Math.floor(n / 2);
  const recent = slice.slice(half), prior = slice.slice(0, half);
  const rXGF = recent.length ? recent.reduce((s, g) => s + xGFkey(g), 0) / recent
  const pXGF = prior.length  ? prior.reduce((s, g)  => s + xGFkey(g), 0) / prior.
  const trendDir = rXGF > pXGF + 0.05 ? 1 : rXGF < pXGF - 0.05 ? -1 : 0;
  const trendMag = pXGF > 0 ? Math.abs(rXGF - pXGF) / pXGF : 0;

  const rXGA = recent.length ? recent.reduce((s, g) => s + xGAkey(g), 0) / recent
  const pXGA = prior.length  ? prior.reduce((s, g)  => s + xGAkey(g), 0) / prior.
  const defTrend = xGA > 0 ? (rXGA - pXGA) / xGA : 0;

  const homeG = slice.filter(g => (g.home_or_away || "").toUpperCase() === "HOME"
  const awayG = slice.filter(g => (g.home_or_away || "").toUpperCase() === "AWAY"
  const homeXGF = homeG.length ? homeG.reduce((s, g) => s + xGFkey(g), 0) / homeG
  const awayXGF = awayG.length ? awayG.reduce((s, g) => s + xGFkey(g), 0) / awayG

  return {
    n, xGF, xGA, hdXGF, hdXGA, xGF_pp,
    trendDir, trendMag, defTrend, homeXGF, awayXGF,
    confidence: Math.min(1, n / (ROLLING_W * 1.5)),
  };
}

// ── Prediction engine — 8-layer signal stack ──────────────────────────
function predictGame(hP, aP, hG, aG, ctx = {}) {
  if (!hP || !aP) return null;

  // 1 — Base lambda (Dixon-Coles xG)
  const hDefQ = aP.xGA > 0 ? aP.xGA : LEAGUE_XGF_AVG;
  const aDefQ = hP.xGA > 0 ? hP.xGA : LEAGUE_XGF_AVG;
```

```
let lH = (hP.homeXGF > 0 ? hP.homeXGF : LEAGUE_XGF_AVG) * (hDefQ / LEAGUE_XGF_A
let lA = (aP.awayXGF > 0 ? aP.awayXGF : LEAGUE_XGF_AVG) * (aDefQ / LEAGUE_XGF_A

// 2 — High-danger xG ratio
const hdH = hP.hdXGF > 0 && hP.xGF > 0 ? hP.hdXGF / hP.xGF : LEAGUE_HD_RATIO;
const hdA = aP.hdXGF > 0 && aP.xGF > 0 ? aP.hdXGF / aP.xGF : LEAGUE_HD_RATIO;
lH *= 1 + (hdH - LEAGUE_HD_RATIO) * 0.4;
lA *= 1 + (hdA - LEAGUE_HD_RATIO) * 0.4;

// 3 — Defensive drift
if (hP.defTrend > 0.05) lA *= 1 + hP.defTrend * 0.15;
if (aP.defTrend > 0.05) lH *= 1 + aP.defTrend * 0.15;

// 4 — Offensive trend
if (hP.trendDir ===  1) lH *= 1 + hP.trendMag * 0.12;
if (hP.trendDir === -1) lH *= 1 - hP.trendMag * 0.09;
if (aP.trendDir ===  1) lA *= 1 + aP.trendMag * 0.12;
if (aP.trendDir === -1) lA *= 1 - aP.trendMag * 0.09;

// 5 — Special teams PP contribution
lH += hP.xGF_pp * 0.28;
lA += aP.xGF_pp * 0.28;

// 6 — B2B fatigue
if (ctx.b2bHome) lH *= 0.93;
if (ctx.b2bAway) lA *= 0.93;

// 7 & 8 — GOALIE xSV% + form trend (the new signal)
// Hot home goalie → away team's lambda DOWN; cold home goalie → UP
if (hG) {
  lA = Math.max(1.0, lA - hG.lambdaMod);
  if (hG.formDir ===  1) lA *= 0.96;
  if (hG.formDir === -1) lA *= 1.04;
}
if (aG) {
  lH = Math.max(1.0, lH - aG.lambdaMod);
  if (aG.formDir ===  1) lH *= 0.96;
  if (aG.formDir === -1) lH *= 1.04;
}

lH = Math.max(1.2, Math.min(5.5, lH));
lA = Math.max(1.2, Math.min(5.5, lA));

const { pHomeWin, pAwayWin } = Poisson.winProbs(lH, lA);
const eT  = lH + lA;
const line = Math.round(eT * 2) / 2;
const { pOver, pUnder } = Poisson.totalProbs(lH, lA, line);
```

```
  const conf = Math.min(hP.confidence, aP.confidence);

  return {
    lH: Math.round(lH * 100) / 100,
    lA: Math.round(lA * 100) / 100,
    pHomeWin: Math.round(pHomeWin * 1000) / 10,
    pAwayWin: Math.round(pAwayWin * 1000) / 10,
    expectedTotal: Math.round(eT * 100) / 100,
    inferredLine: line,
    pOver:  Math.round(pOver  * 1000) / 10,
    pUnder: Math.round(pUnder * 1000) / 10,
    mlPick: pHomeWin >= pAwayWin ? "HOME" : "AWAY",
    mlConf: Math.round(Math.max(pHomeWin, pAwayWin) * 10) / 10,
    ouPick: pOver >= pUnder ? "OVER" : "UNDER",
    ouConf: Math.round(Math.max(pOver, pUnder) * 10) / 10,
    conf: Math.round(conf * 100),
    goalieImpact: {
      home: hG ? { lambdaMod: Math.round(hG.lambdaMod*100)/100, xSvDelta: Math.ro
      away: aG ? { lambdaMod: Math.round(aG.lambdaMod*100)/100, xSvDelta: Math.ro
    },
    hP, aP,
  };
}

// ── Backtester ──────────────────────────────────────────────────
function runBacktest(teamRows, goalieIdx, onProgress) {
  const teamGames = buildTeamProfiles(teamRows);

  // Build game index from team rows
  const gameIndex = {};
  for (const r of teamRows) {
    const gid = r.gameId || r.game_id; if (!gid) continue;
    const ha = (r.home_or_away || "").toUpperCase();
    const team = r.team || r.playerTeam;
    if (!gameIndex[gid]) gameIndex[gid] = { date: sf(r.gameDate || r.date), seaso
    if (ha === "HOME") { gameIndex[gid].home = team; gameIndex[gid].goalsH = sf(r
    if (ha === "AWAY") { gameIndex[gid].away = team; gameIndex[gid].goalsA = sf(r
  }

  const gameIds = Object.keys(gameIndex).sort((a, b) => gameIndex[a].date - gameI
  const blankSt = () => ({ mlC:0, mlT:0, ouC:0, ouT:0, roiML:0, roiOU:0, stakeML:
  const st = { with: blankSt(), without: blankSt() };
  const results = [];
  let processed = 0;

  for (const gid of gameIds) {
    const g = gameIndex[gid];
```

```
if (!g.home || !g.away || isNaN(g.goalsH) || isNaN(g.goalsA) || g.goalsH == n

const hGames = teamGames[g.home] || [];
const aGames = teamGames[g.away] || [];
const hIdx   = hGames.findIndex(r => (r.gameId || r.game_id) === gid);
const aIdx   = aGames.findIndex(r => (r.gameId || r.game_id) === gid);
if (hIdx < MIN_GAMES || aIdx < MIN_GAMES) continue;

const hProf = rollingTeamStats(hGames, hIdx, ROLLING_W);
const aProf = rollingTeamStats(aGames, aIdx, ROLLING_W);
if (!hProf || !aProf) continue;

const hGoalie = goalieIdx ? goalieForm(goalieIdx, g.home, gid) : null;
const aGoalie = goalieIdx ? goalieForm(goalieIdx, g.away, gid) : null;

const predWith    = predictGame(hProf, aProf, hGoalie, aGoalie);
const predWithout = predictGame(hProf, aProf, null, null);
if (!predWith || !predWithout) continue;

const actualWinner = g.goalsH > g.goalsA ? "HOME" : g.goalsA > g.goalsH ? "AW
const actualTotal  = g.goalsH + g.goalsA;
const actualOU     = actualTotal > predWith.inferredLine ? "OVER"
                   : actualTotal < predWith.inferredLine ? "UNDER" : "PUSH";

for (const [key, pred] of [["with", predWith], ["without", predWithout]]) {
  const s = st[key];
  if (pred.mlConf > 55 && actualWinner !== "PUSH") {
    const ok = pred.mlPick === actualWinner;
    s.mlC += ok ? 1 : 0; s.mlT++;
    s.stakeML += STAKE;
    s.roiML   += ok ? STAKE * (100 / JUICE) : -STAKE;
  }
  if (pred.ouConf > 55 && actualOU !== "PUSH") {
    const ok = pred.ouPick === actualOU;
    s.ouC += ok ? 1 : 0; s.ouT++;
    s.stakeOU += STAKE;
    s.roiOU   += ok ? STAKE * (100 / JUICE) : -STAKE;
  }
}

results.push({
  gid, season: g.season, date: g.date,
  home: g.home, away: g.away,
  scoreH: g.goalsH, scoreA: g.goalsA,
  actualWinner, actualTotal, actualOU,
  predWith, predWithout,
  hGoalie, aGoalie,
```

```
      hasGoalie: !!(hGoalie || aGoalie),
      mlCorrect: predWith.mlPick === actualWinner,
      ouCorrect: actualOU !== "PUSH" && predWith.ouPick === actualOU,
    });

    if (++processed % 500 === 0) onProgress?.(processed, gameIds.length);
    if (processed >= 20000) break;
  }

  const fmt = s => ({
    mlWR:  s.mlT    ? (s.mlC / s.mlT * 100).toFixed(1) : "—",
    ouWR:  s.ouT    ? (s.ouC / s.ouT * 100).toFixed(1) : "—",
    mlROI: s.stakeML ? (s.roiML / s.stakeML * 100).toFixed(2) : "—",
    ouROI: s.stakeOU ? (s.roiOU / s.stakeOU * 100).toFixed(2) : "—",
    mlPL: Math.round(s.roiML), ouPL: Math.round(s.roiOU),
    mlBets: s.mlT, ouBets: s.ouT, ...s,
  });

  return { results, withGoalie: fmt(st.with), withoutGoalie: fmt(st.without) };
}

// ── UI helpers ──────────────────────────────────────────────────
const C = {
  bg:"#0b0d14",bg2:"#0f1220",bg3:"#141728",bg4:"#1a1f35",
  border:"rgba(255,255,255,0.07)",border2:"rgba(255,255,255,0.12)",
  green:"#3de89e",red:"#f06470",yellow:"#f5c518",blue:"#6ab0f5",
  orange:"#f0956b",purple:"#a78bfa",
  text:"#dde3ee",text2:"#8891a8",text3:"#3d4560",
};
const mono = { fontFamily:"monospace" };
const Tag = ({ label, color=C.text2, size=9 }) => (
  <span style={{...mono,fontSize:size,letterSpacing:1.2,padding:"2px 7px",borderR
    background:color+"18",color,border:`1px solid ${color}33`}}>{label}</span>
);
const Stat = ({ label, value, color=C.text, sub, accent }) => (
  <div style={{background:C.bg3,border:`1px solid ${accent?accent+"44":C.border}`
    borderTop:accent?`2px solid ${accent}`:undefined,borderRadius:7,padding:"12px
    <div style={{...mono,fontSize:9,color:C.text3,letterSpacing:1.5,textTransform
    <div style={{...mono,fontSize:20,fontWeight:700,color,lineHeight:1}}>{value}<
    {sub&&<div style={{...mono,fontSize:9,color:C.text3,marginTop:4}}>{sub}</div>
  </div>
);
const CompRow = ({ label, w, wo, higher=true }) => {
  const wn=parseFloat(w), won=parseFloat(wo);
  const better = higher ? wn>won : wn<won;
  const delta = isNaN(wn)||isNaN(won) ? null : wn-won;
  return (
```

```
    <div style={{display:"grid",gridTemplateColumns:"1fr 110px 110px 70px",gap:8,
      borderBottom:`1px solid ${C.border}`,alignItems:"center"}}>
      <span style={{fontSize:12,color:C.text2}}>{label}</span>
      <span style={{...mono,fontSize:13,fontWeight:700,color:better?C.green:C.red
      <span style={{...mono,fontSize:13,color:C.text2,textAlign:"right"}}>{wo}</s
      <span style={{...mono,fontSize:11,color:delta==null?C.text3:better?C.green:
        {delta!=null?(delta>0?"+":"")+delta.toFixed(2):"—"}
      </span>
    </div>
  );
};

const GoalieChip = ({ g, label }) => {
  if (!g) return <div style={{background:C.bg4,border:`1px solid ${C.border}`,bor
    <div style={{...mono,fontSize:9,color:C.text3,marginBottom:4}}>{label} GOALIE
    <span style={{...mono,fontSize:10,color:C.text3}}>No data — load goalie CSV</
  </div>;
  const fc = g.formDir===1?C.green:g.formDir===-1?C.red:C.text3;
  return (
    <div style={{background:C.bg4,border:`1px solid ${C.purple}33`,borderRadius:6
      <div style={{...mono,fontSize:9,color:C.purple,marginBottom:6,letterSpacing
      <div style={{display:"flex",gap:6,flexWrap:"wrap"}}>
        <Tag label={`xSV% ${g.xSvPct}%`} color={g.xSvPct>LEAGUE_XSV_PCT*100?C.gre
        <Tag label={`GSAE ${g.xSvDelta>0?"+":""}${g.xSvDelta}`} color={g.xSvDelta
        <Tag label={`λ adj ${g.lambdaMod>0?"-":"+"}${Math.abs(g.lambdaMod)}`} col
        <Tag label={g.formDir===1?"▲ HOT":g.formDir===-1?"▼ COLD":"FLAT"} color={
      </div>
    </div>
  );
};

// ── Main App ──────────────────────────────────────────────────────
export default function App() {
  const [phase, setPhase]       = useState("setup");
  const [tab,   setTab]         = useState("compare");
  const [prog,  setProg]        = useState({ msg:"", pct:0 });
  const [bt,    setBt]          = useState(null);
  const [teamRows, setTeamRows] = useState(null);
  const [gIdx,  setGIdx]        = useState(null);
  const [fileNames, setFileNames] = useState({ teams:"", goalies:"" });
  const [error, setError]       = useState("");
  const [liveT, setLiveT]       = useState({ home:"", away:"" });
  const [livePred, setLivePred] = useState(null);

  const teamList = teamRows
    ? [...new Set(teamRows.map(r=>r.team||r.playerTeam).filter(Boolean))].sort()
```

```
const loadTeams = useCallback(async file => {
  if (!file) return; setError("");
  setProg({ msg:`Reading ${file.name}…`, pct:10 });
  try {
    const rows = parseCSV(await file.text());
    const s = rows[0]||{};
    if (!("xGoals" in s || "xGF_5v5" in s || "xGF" in s))
      return setError(`Columns: ${Object.keys(s).slice(0,8).join(", ")} — expec
    setTeamRows(rows);
    setFileNames(f=>({...f, teams:file.name}));
    setProg({ msg:`✓ ${rows.length.toLocaleString()} team rows loaded`, pct:100
  } catch(e) { setError(e.message); }
}, []);

const loadGoalies = useCallback(async file => {
  if (!file) return; setError("");
  setProg({ msg:`Reading ${file.name}…`, pct:10 });
  try {
    const rows = parseCSV(await file.text());
    const gRows = rows.filter(r=>(r.position||"").toUpperCase()==="G" && r.situ
    if (!gRows.length) return setError("No goalie rows found. Need gameByGame g
    const idx = buildGoalieIndex(gRows);
    setGIdx(idx);
    setFileNames(f=>({...f, goalies:file.name}));
    setProg({ msg:`✓ Goalie index: ${gRows.length.toLocaleString()} starts acro
  } catch(e) { setError(e.message); }
}, []);

const runAnalysis = useCallback(async () => {
  if (!teamRows) return setError("Load team file first");
  setError(""); setPhase("loading");
  setProg({ msg:"Building rolling xG profiles…", pct:5 });
  await new Promise(r=>setTimeout(r,30));
  try {
    const result = runBacktest(teamRows, gIdx, (done, total) =>
      setProg({ msg:`Backtesting: ${done.toLocaleString()} games…`, pct:5+Math.
    );
    setBt(result); setPhase("done"); setTab("compare");
  } catch(e) { setError(e.message); setPhase("setup"); }
}, [teamRows, gIdx]);

function livePrediction() {
  if (!teamRows || !liveT.home || !liveT.away) return;
  const tp = buildTeamProfiles(teamRows);
  const hG = tp[liveT.home]||[], aG = tp[liveT.away]||[];
  if (!hG.length || !aG.length) return;
  const hProf = rollingTeamStats(hG, hG.length, ROLLING_W);
```

```
  const aProf = rollingTeamStats(aG, aG.length, ROLLING_W);
  const hGoalie = gIdx ? goalieForm(gIdx, liveT.home, hG[hG.length-1]?.gameId)
  const aGoalie = gIdx ? goalieForm(gIdx, liveT.away, aG[aG.length-1]?.gameId)
  setLivePred(predictGame(hProf, aProf, hGoalie, aGoalie));
}

function DropZone({ id, label, sub, loaded, onFile, accent=C.green }) {
  return (
    <div onClick={()=>document.getElementById(id).click()}
      onDragOver={e=>e.preventDefault()}
      onDrop={e=>{e.preventDefault();onFile(e.dataTransfer.files[0]);}}
      onMouseOver={e=>e.currentTarget.style.borderColor=accent}
      onMouseOut={e=>e.currentTarget.style.borderColor=loaded?accent:C.border}
      style={{border:`2px dashed ${loaded?accent:C.border}`,borderRadius:10,
        padding:"26px 22px",textAlign:"center",cursor:"pointer",
        background:loaded?accent+"08":"transparent",transition:"border-color .2
      <input id={id} type="file" accept=".csv" style={{display:"none"}} onChang
      <div style={{fontSize:26,marginBottom:8}}>{loaded?"✅":"📂"}</div>
      <div style={{...mono,fontSize:11,color:loaded?accent:C.text2,marginBottom
      <div style={{fontSize:11,color:C.text3}}>{sub}</div>
    </div>
  );
}

return (
  <div style={{minHeight:"100vh",background:C.bg,color:C.text,fontFamily:"'DM S
    <style>{`
      @import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;50
      *{box-sizing:border-box;margin:0;padding:0}
      ::-webkit-scrollbar{width:4px;height:4px}::-webkit-scrollbar-thumb{backgr
      @keyframes spin{to{transform:rotate(360deg)}}
      @keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;t
      select{background:${C.bg3};border:1px solid ${C.border};border-radius:5px
    `}</style>

    {/* HEADER */}
    <div style={{background:C.bg2,borderBottom:`1px solid ${C.border}`,padding:
      display:"flex",alignItems:"center",justifyContent:"space-between"}}>
      <div>
        <div style={{display:"flex",alignItems:"baseline",gap:12}}>
          <span style={{...mono,fontSize:17,fontWeight:700,letterSpacing:3,colo
          <span style={{...mono,fontSize:10,color:C.text3}}>xG + Goalie xSV% En
        </div>
        <div style={{display:"flex",gap:8,marginTop:6}}>
          {fileNames.teams   && <Tag label={`TEAMS: ${fileNames.teams}`}   colo
          {fileNames.goalies && <Tag label={`GOALIES: ${fileNames.goalies}`} co
          {!fileNames.goalies && <Tag label="GOALIE xSV% — load CSV to activate
```

```
    </div>
  </div>
  {phase==="done"&&bt&&(
    <div style={{display:"flex",gap:8}}>
      <Tag label={`${bt.results.length.toLocaleString()} GAMES`} color={C.t
      <Tag label={`ML ${bt.withGoalie.mlWR}% WR`} color={parseFloat(bt.with
      <Tag label={`ROI ${bt.withGoalie.mlROI}%`} color={parseFloat(bt.withG
    </div>
  )}
</div>

{/* LOADING */}
{phase==="loading"&&(
  <div style={{padding:"80px 28px",textAlign:"center",animation:"fadeIn .3s
    <div style={{width:44,height:44,borderRadius:"50%",border:`3px solid ${
      borderTop:`3px solid ${C.green}`,animation:"spin 1s linear infinite",
    <div style={{...mono,fontSize:12,color:C.text2,marginBottom:14}}>{prog.
    <div style={{width:320,height:4,background:C.bg3,borderRadius:2,margin:
      <div style={{height:"100%",background:C.green,width:`${prog.pct}%`,tr
    </div>
  </div>
)}

{/* SETUP */}
{phase==="setup"&&(
  <div style={{padding:"28px",maxWidth:880,margin:"0 auto",animation:"fadeI
    {error&&(
      <div style={{background:"#250a0a",border:`1px solid ${C.red}44`,borde
        padding:"12px 16px",marginBottom:18,...mono,fontSize:11,color:C.red
    )}

    {/* What's new */}
    <div style={{background:C.purple+"0d",border:`1px solid ${C.purple}33`,
      borderRadius:8,padding:"14px 18px",marginBottom:22}}>
      <div style={{...mono,fontSize:10,color:C.purple,letterSpacing:1.5,mar
      <div style={{fontSize:12,color:C.text2,lineHeight:1.85}}>
        A goalie averaging <strong style={{color:C.text}}>+0.5 GSAE (Goals
        his last 5 games now reduces the opposing lambda by ~0.125 expected
        The engine runs a blind A/B test every backtest so you see the exac
        <strong style={{color:C.green}}> Load the goalie CSV below to activ
      </div>
    </div>

    {/* Drop zones */}
    <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:14,margin
      <div>
        <div style={{...mono,fontSize:9,color:C.text3,letterSpacing:1.5,mar
```

```
    <DropZone id="fi-teams" label="all_teams.csv"
      sub="moneypuck.com/data.htm → Game By Game → all teams"
      onFile={loadTeams} loaded={!!fileNames.teams} accent={C.green} />
  </div>
  <div>
    <div style={{...mono,fontSize:9,color:C.purple,letterSpacing:1.5,ma
    <DropZone id="fi-goalies" label="Goalie game-by-game CSV"
      sub="moneypuck.com/data.htm → Game By Game → Goalies → All Previo
      onFile={loadGoalies} loaded={!!fileNames.goalies} accent={C.purpl
  </div>
</div>

{prog.msg&&!error&&(
  <div style={{...mono,fontSize:11,color:C.green,marginBottom:16,paddin
    background:C.green+"0a",border:`1px solid ${C.green}22`,borderRadiu
)}

<button onClick={runAnalysis} disabled={!teamRows} style={{
  ...mono,fontWeight:700,fontSize:12,letterSpacing:2,padding:"13px 32px
  background:teamRows?C.green:C.bg3, color:teamRows?"#0b1a10":C.text3,
  border:`1px solid ${teamRows?C.green:C.border}`,
  borderRadius:8,cursor:teamRows?"pointer":"not-allowed",width:"100%",m
}}>
  {!teamRows ? "LOAD all_teams.csv FIRST"
    : gIdx ? "▶ RUN BACKTEST — xG + Goalie xSV% (Full 8-Signal Stack)"
    : "▶ RUN BACKTEST — xG Only (Load goalie CSV for +ROI)"}
</button>

{/* Signal stack */}
<div style={{background:C.bg2,border:`1px solid ${C.border}`,borderRadi
  <div style={{...mono,fontSize:10,color:C.text2,letterSpacing:2,margin
  <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:10}}>
    {[
      {n:"01",label:"xGF_5v5 rolling 10G",       impact:"Primary attack
      {n:"02",label:"xGA_5v5 rolling 10G",       impact:"Primary defens
      {n:"03",label:"High-danger xG ratio",      impact:"Shot quality w
      {n:"04",label:"Offensive trend (5G vs 5G)","impact":"Momentum adj
      {n:"05",label:"Defensive drift (xGA)",     impact:"Defense gettin
      {n:"06",label:"PP xGF contribution",       impact:"Special teams
      {n:"07",label:"Goalie GSAE rolling 5 starts","impact":"±0.5 lambd
      {n:"08",label:"Goalie form trend (3G)",    impact:"Hot/cold strea
    ].map(s=>(
      <div key={s.n} style={{display:"flex",gap:10,alignItems:"flex-sta
        padding:"8px 0",borderBottom:`1px solid ${C.border}`}}>
        <span style={{...mono,fontSize:10,color:C.text3,minWidth:22}}>{
        <div style={{flex:1}}>
          <div style={{fontSize:12,fontWeight:600,color:C.text}}>{s.lab
```

```
              <div style={{fontSize:11,color:C.text3,marginTop:2}}>{s.impac
            </div>
            <Tag label={s.on?"ACTIVE":"LOAD CSV"} color={s.on?s.color:C.yel
          </div>
        ))}
      </div>
    </div>
  </div>
)}

{/* RESULTS */}
{phase==="done"&&bt&&(
  <div style={{padding:"20px 28px",animation:"fadeIn .3s ease"}}>
    {/* TABS */}
    <div style={{display:"flex",borderBottom:`1px solid ${C.border}`,gap:2,
      {["compare","game log","predictor","goalie impact"].map(t=>(
        <button key={t} onClick={()=>setTab(t)} style={{
          ...mono,fontSize:10,letterSpacing:1.5,padding:"9px 18px",backgrou
          borderBottom:tab===t?`2px solid ${C.green}`:"2px solid transparen
          color:tab===t?C.green:C.text2,cursor:"pointer",textTransform:"upp
        }}>{t}</button>
      ))}
    </div>

    {/* COMPARE */}
    {tab==="compare"&&(
      <div>
        {/* System cards */}
        <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:14,ma
          {[
            {title:"WITH Goalie xSV%",  stats:bt.withGoalie,    color:C.gre
            {title:"WITHOUT Goalie (control)",stats:bt.withoutGoalie,color:
          ].map(sys=>(
            <div key={sys.title} style={{background:C.bg2,border:`1px solid
              borderTop:`3px solid ${sys.color}`,borderRadius:10,padding:20
              <div style={{...mono,fontSize:10,color:sys.color,letterSpacin
              <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap
                <Stat label="ML Bets" value={sys.stats.mlBets} color={C.tex
                <Stat label="ML Win Rate" value={`${sys.stats.mlWR}%`}
                  color={parseFloat(sys.stats.mlWR)>55?C.green:C.yellow} />
                <Stat label="ML P&L" value={`${sys.stats.mlPL>=0?"+":""}$${
                  color={sys.stats.mlPL>0?C.green:C.red} />
                <Stat label="ML ROI" value={`${parseFloat(sys.stats.mlROI)>
                  color={parseFloat(sys.stats.mlROI)>0?C.green:C.red}
                  accent={parseFloat(sys.stats.mlROI)>0?C.green:C.red} />
                <Stat label="O/U Win Rate" value={`${sys.stats.ouWR}%`}
                  color={parseFloat(sys.stats.ouWR)>55?C.green:C.yellow} />
```

```
        <Stat label="O/U ROI" value={`${parseFloat(sys.stats.ouROI)
          color={parseFloat(sys.stats.ouROI)>0?C.green:C.red} />
      </div>
    </div>
  ))}
</div>

{/* Delta table */}
<div style={{background:C.bg2,border:`1px solid ${C.border}`,border
  <div style={{display:"grid",gridTemplateColumns:"1fr 110px 110px
    {["METRIC","WITH GOALIE","WITHOUT","DELTA"].map((h,i)=>(
      <span key={h} style={{...mono,fontSize:9,color:i===0?C.text3:
        textAlign:i>0?"right":"left",letterSpacing:1}}>{h}</span>
    ))}
  </div>
  <CompRow label="ML Win Rate %"     w={bt.withGoalie.mlWR}  wo={bt
  <CompRow label="ML ROI %"           w={bt.withGoalie.mlROI} wo={b
  <CompRow label="O/U Win Rate %"     w={bt.withGoalie.ouWR}  wo={b
  <CompRow label="O/U ROI %"          w={bt.withGoalie.ouROI} wo={
</div>

{/* Verdict */}
{(()=>{
  const wROI = parseFloat(bt.withGoalie.mlROI);
  const woROI = parseFloat(bt.withoutGoalie.mlROI);
  const lift = isNaN(wROI)||isNaN(woROI) ? null : wROI-woROI;
  const color = wROI>2?C.green:wROI>0?C.yellow:C.red;
  return (
    <div style={{background:color+"0a",border:`1px solid ${color}33
      <div style={{...mono,fontSize:10,color,letterSpacing:2,margin
        {gIdx ? "GOALIE xSV% IMPACT — VERIFIED BY BACKTEST" : "VERD
      </div>
      <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap
        <div style={{fontSize:12,color:C.text2,lineHeight:1.9}}>
          {gIdx ? (
            lift > 0 ? <>✅ <strong style={{color:C.green}}>+{lift.
              GSAE per start is predictive. Teams with hot goalies
            </> : <>⚠️ Goalie data loaded, delta is {lift?.toFixed(
              Short windows can show mean-reversion. Try reviewing
            </>
          ) : <>Load the MoneyPuck goalie game-by-game CSV to see t
            Projected lift: <strong style={{color:C.green}}>+3–5%</
          </>}
        </div>
        <div style={{fontSize:12,color:C.text2,lineHeight:1.9}}>
          <strong style={{color:C.text}}>Calibration:</strong> +0.5
          → lambdaMod = +0.125 → opposing team scores ~0.125 fewer
```

```
                → shifts ML win probability ~2-4 points. Form trend layer
              </div>
            </div>
          </div>
        );
      })()}
    </div>
  )}

  {/* GAME LOG */}
  {tab==="game log"&&(
    <div style={{background:C.bg2,border:`1px solid ${C.border}`,borderRa
      <div style={{display:"grid",gridTemplateColumns:"60px 1fr 65px 90px
        gap:8,padding:"8px 14px",background:"rgba(0,0,0,.3)"}}>
        {["Date","Game","Score","Pred","ML","O/U","Goalie λ mod"].map(h=>
          <span key={h} style={{...mono,fontSize:9,color:C.text3,letterSp
        ))}
      </div>
      {bt.results.slice(0,300).map((r,i)=>(
        <div key={i} style={{display:"grid",gridTemplateColumns:"60px 1fr
          gap:8,padding:"7px 14px",borderBottom:`1px solid ${C.border}`,a
          background:r.mlCorrect?C.green+"05":"transparent"}}>
          <span style={{...mono,fontSize:10,color:C.text3}}>{String(r.dat
          <span style={{fontSize:12,fontWeight:600}}>{r.away}@{r.home}</s
          <span style={{...mono,fontSize:11,color:C.text2}}>{r.scoreA}-{r
          <span style={{...mono,fontSize:10,color:r.predWith.mlPick==="HO
            {r.predWith.mlPick} {r.predWith.mlConf}%
          </span>
          <span style={{...mono,fontSize:11,color:r.mlCorrect?C.green:C.r
            {r.mlCorrect?"✓ WIN":"✗ LOSS"}
          </span>
          <span style={{...mono,fontSize:10,color:r.ouCorrect?C.green:C.r
            {r.predWith.ouPick} {r.ouCorrect?"✓":"✗"}
          </span>
          <div style={{display:"flex",gap:4,flexWrap:"wrap"}}>
            {r.hGoalie&&<Tag label={`H ${r.hGoalie.lambdaMod>0?"-":"+"}${
            {r.aGoalie&&<Tag label={`A ${r.aGoalie.lambdaMod>0?"-":"+"}${
            {!r.hasGoalie&&<span style={{...mono,fontSize:9,color:C.text3
          </div>
        </div>
      ))}
      {bt.results.length>300&&(
        <div style={{padding:"10px 14px",...mono,fontSize:10,color:C.text
          Showing 300 of {bt.results.length.toLocaleString()}
        </div>
      )}
    </div>
```

```
)}

{/* PREDICTOR */}
{tab==="predictor"&&(
  <div style={{maxWidth:680}}>
    <div style={{background:C.bg2,border:`1px solid ${C.border}`,border
      <div style={{...mono,fontSize:10,color:C.text2,letterSpacing:2,ma
        LIVE PREDICTION — full 8-signal stack on current rolling data
      </div>
      <div style={{display:"flex",gap:12,alignItems:"flex-end",marginBo
        {["away","home"].map(side=>(
          <div key={side} style={{flex:1}}>
            <div style={{...mono,fontSize:9,color:C.text3,marginBottom:
            <select value={liveT[side]} onChange={e=>setLiveT(t=>({...t
              <option value="">Select…</option>
              {teamList.map(t=><option key={t} value={t}>{t}</option>)}
            </select>
          </div>
        ))}
        <button onClick={livePrediction} disabled={!liveT.home||!liveT.
          ...mono,fontSize:11,fontWeight:700,padding:"9px 18px",
          background:liveT.home&&liveT.away?C.green:C.bg3,
          color:liveT.home&&liveT.away?"#0a1a0e":C.text3,
          border:"none",borderRadius:6,cursor:"pointer",whiteSpace:"now
        }}>PREDICT</button>
      </div>

      {livePred&&(
        <div style={{animation:"fadeIn .3s ease"}}>
          <div style={{display:"grid",gridTemplateColumns:"repeat(3,1fr
            <Stat label="ML Pick" value={livePred.mlPick} color={C.gree
            <Stat label="Expected Total" value={livePred.expectedTotal}
            <Stat label="O/U Pick" value={`${livePred.ouPick} ${livePre
          </div>
          <div style={{display:"grid",gridTemplateColumns:"repeat(4,1fr
            <Stat label="λ Home" value={livePred.lH} color={C.blue} sub
            <Stat label="λ Away" value={livePred.lA} color={C.orange} s
            <Stat label="P(Home Win)" value={`${livePred.pHomeWin}%`} c
            <Stat label="P(Away Win)" value={`${livePred.pAwayWin}%`} c
          </div>
          <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap
            <GoalieChip g={livePred.goalieImpact?.home} label="HOME" />
            <GoalieChip g={livePred.goalieImpact?.away} label="AWAY" />
          </div>
        </div>
      )}
    </div>
```

```
  </div>
)}

{/* GOALIE IMPACT */}
{tab==="goalie impact"&&(
  <div>
    {!gIdx ? (
      <div style={{background:C.bg2,border:`1px solid ${C.yellow}33`,bo
        <div style={{fontSize:36,marginBottom:16}}>📖</div>
        <div style={{...mono,fontSize:12,color:C.yellow,marginBottom:12
        <div style={{fontSize:12,color:C.text2,maxWidth:480,margin:"0 a
          Go to Setup and drop the MoneyPuck goalie game-by-game CSV.
          Download at moneypuck.com/data.htm → Game By Game Level Data
          The engine will immediately show you the ROI lift season by s
        </div>
      </div>
    ) : (()=>{
      const bySeason = {};
      for (const r of bt.results) {
        if (!r.hasGoalie) continue;
        const s = r.season||"?";
        if (!bySeason[s]) bySeason[s]={wC:0,wT:0,woC:0,woT:0};
        const bs=bySeason[s];
        if (r.predWith.mlConf>55&&r.actualWinner!=="PUSH") {
          bs.wC+=r.predWith.mlPick===r.actualWinner?1:0; bs.wT++;
        }
        if (r.predWithout.mlConf>55&&r.actualWinner!=="PUSH") {
          bs.woC+=r.predWithout.mlPick===r.actualWinner?1:0; bs.woT++;
        }
      }
      const seasons=Object.keys(bySeason).filter(s=>bySeason[s].wT>=10)
      const totalWith    = seasons.reduce((s,k)=>s+bySeason[k].wC,0);
      const totalWithT   = seasons.reduce((s,k)=>s+bySeason[k].wT,0);
      const totalWithout = seasons.reduce((s,k)=>s+bySeason[k].woC,0);
      const totalWithoutT= seasons.reduce((s,k)=>s+bySeason[k].woT,0);
      const overallWith  = totalWithT  ? (totalWith/totalWithT*100).toF
      const overallWithout=totalWithoutT?(totalWithout/totalWithoutT*10
      const overallLift  = isNaN(parseFloat(overallWith))||isNaN(parseF
        : (parseFloat(overallWith)-parseFloat(overallWithout));
      return (
        <div>
          <div style={{display:"grid",gridTemplateColumns:"repeat(3,1fr
            <Stat label="With Goalie WR (all seasons)" value={`${overal
              color={parseFloat(overallWith)>55?C.green:C.yellow} accen
            <Stat label="Without Goalie WR" value={`${overallWithout}%`
            <Stat label="Overall Lift" value={overallLift!=null?`${over
              color={overallLift>0?C.green:C.red} accent={overallLift>0
```

```
                    </div>
                    <div style={{background:C.bg2,border:`1px solid ${C.border}`,
                      <div style={{padding:"11px 16px",borderBottom:`1px solid ${
                        ...mono,fontSize:10,color:C.text2,letterSpacing:2}}>
                        SEASON-BY-SEASON GOALIE xSV% LIFT (goalie-enriched games
                      </div>
                      <div style={{display:"grid",gridTemplateColumns:"70px 120px
                        gap:8,padding:"8px 16px",background:"rgba(0,0,0,.2)"}}>
                        {["Season","With Goalie","Without","Bar","Lift"].map(h=>(
                          <span key={h} style={{...mono,fontSize:9,color:C.text3,
                        ))}
                      </div>
                      {seasons.map(s=>{
                        const d=bySeason[s];
                        const w =d.wT  ? d.wC/d.wT*100  : 0;
                        const wo=d.woT ? d.woC/d.woT*100 : 0;
                        const lift=w-wo;
                        return (
                          <div key={s} style={{display:"grid",gridTemplateColumns
                            gap:8,padding:"8px 16px",borderBottom:`1px solid ${C.
                            <span style={{...mono,fontSize:11,color:C.text3}}>{s}
                            <span style={{...mono,fontSize:12,fontWeight:700,colo
                            <span style={{...mono,fontSize:12,color:C.text2}}>{wo
                            <div style={{position:"relative",height:6,background:
                              <div style={{position:"absolute",left:"50%",top:0,h
                                width:`${Math.min(50,Math.abs(lift))}%`,backgroun
                                borderRadius:3,transform:lift>0?"none":"translate
                              <div style={{position:"absolute",left:"50%",top:0,h
                            </div>
                            <span style={{...mono,fontSize:11,fontWeight:700,colo
                              {lift>0?"+":""}{lift.toFixed(1)}%
                            </span>
                          </div>
                        );
                      })}
                    </div>
                  </div>
                );
              })()}
            </div>
          )}
        </div>
      )}
    </div>
  );
}
```